

Git応用編

citrus88

初版: 2025/09/01

更新日: 2025/09/01

はじめに

想定読者

- Gitの基本的な使い方は理解している人

本資料のゴール

Gitで以下の応用的な使い方ができる

- エイリアス設定
 - `git config --global alias.`
- 管理対象のファイルを選択する
 - `.gitignore` ファイル
- タグ付け
 - `git tag`
- 作業中の変更を一時的に退避する
 - `git stash`
- コミットする範囲を指定する
 - `git add -p`
- 直前のコミットへの操作
 - `git commit --amend`
- コミットの整理
 - `git rebase -i`
 - `git reset --soft`
- 別ブランチにある特定のコミットを適用する
 - `git cherry-pick`
- 行単位の変更履歴の確認
 - `git blame`

この資料で説明しないこと

- Gitの基本的な内容

Gitエイリアス設定

エイリアスとは、コマンドなどを短縮するために用いる機能である。

`git config --global alias.<省略名> <コマンド>` という構文で設定することができる。

```
# エイリアス例
# 設定したエイリアスを確認する
git config --global alias.al "config --get-regexp ^alias\."
# 変更状態を確認するコマンド
git config --global alias.st status
# 変更履歴をグラフで可視化するコマンド
# `git lg -5` などとすることで、上位5つまでの履歴を確認できる
git config --global alias.lg "log --oneline --graph"
```

管理対象のファイルを選択する

`.gitignore` を用いることで管理対象外にするファイルを指定できる。`.gitignore` ファイルは、リポジトリが存在するワークディレクトリ内であればどの場所においても良く、`.gitignore` ファイルからの相対パスを用いてファイルを指定する。

`.gitignore` ファイルの書き方にはブラックリスト方式とホワイトリスト方式の2種類がある。ブラックリスト方式は除外したい対象を記載する方法である。一方でホワイトリスト方式は管理したい対象を明示的に指定する方法である。

ブラックリスト方式による管理対象の除外

```
# ブラックリスト方式で、追跡対象から除外する
*.log      # .logと付くファイル全てを指定する
node_modules/ # ディレクトリを追加することも可能
```

ホワイトリスト方式による明示的な管理対象の指定

```
# ホワイトリスト方式で、管理対象を明示的に記載する
# すべてを管理対象から外す
*
# すべてのディレクトリを管理対象に戻す
!*/
# 以後、先頭に`!`をつけて、管理対象を記載する
!.gitignore # .gitignoreを管理対象に戻す
# 管理対象から除外したい場合は、再度記述する
/build      # buildディレクトリをやっぱり管理対象から除外する
```

コミット後の管理対象の指定

すでにコミットしているファイルを後から除外したい場合、以下の対応を行う。ただし、チームで開発している場合、事前に確認・調整が必要である。

ファイルが少量の場合

```
git rm --cached <ファイル名> # `--cached`オプションにより、ファイルはワーキングツリーには残る
git rm -r --cached <ディレクトリ名>
git add .gitignore
git commit -m "remove files from tracking and add to .gitignore"
```

ファイルが大量にある場合

```
# .gitignoreパターンに一致するファイルを一括削除
git ls-files -i c --exclude-standard | xargs git rm --cached
git add .gitignore
git commit -m "remove files from tracking and add to .gitignore"
```

タグ付け(`git tag`)

GitではコミットIDでバージョン管理を行うが、コミットIDはSHA-1形式で人間にとって読みにくいので、タグを付与することができる。例えば、リリースバージョンにはv1.0.0やv2.0.0のようにバージョン番号を付与することができる。

```
# 軽量タグの作成
```

```
git tag v1.0.0
```

```
# 注釈付きタグの作成
```

```
git tag -a v1.0.0 -m "リリース版 1.0.0"
```

```
# タグをリモートにプッシュ
```

```
git push origin v1.0.0
```

```
# すべてのタグをプッシュ
```

```
git push origin --tags
```

作業中の変更を一時的に退避する(`git stash`)

`git stash`を使うことで作業中の変更を一時的に退避することができる。例えば以下のような場面で使用する。

- 緊急のバグ修正で別ブランチに切り替える必要がある
- Pull前に未コミットの変更がある

```
# 変更を退避
# -u (--include-untracked)をつけることで、未追跡ファイルも退避する
git stash -u

# メッセージ付きで退避
git stash save "作業中の変更を一時保存"

# 退避リストを確認
git stash list

# 退避した変更内容に戻す
git stash apply stash@{0} # stash@{0}がない場合、直前に退避した内容に戻す

# 退避している内容を消す
git stash drop stash@{0}

# 退避した変更に戻し、退避した内容を消す
git stash pop stash@{0}
```

コミットする範囲を指定する

一度に多くの箇所を変更してしまった場合、分割してコミットすることができる。その方法は、`git add -p` を用いてステージングエリアに追加する範囲を指定する方法である。

```
git add -p
# 編集画面が出るので操作する
git commit -m "コミット1"
git add -p
# 編集画面が出るので操作する
git commit -m "コミット2"
```

`git add -p` を用いた時の操作方法は以下の通りである。

- `y` を入力すると変更をステージングエリアに追加する
- `n` を入力すると変更をステージングエリアに追加しない
- `s` を入力すると変更を分割してステージングエリアに追加する
- `e` を入力すると変更を編集する
- `q` を入力すると変更をステージングエリアに追加しない

直前のコミットへの操作(`git commit --amend`)

コミット後にコミットメッセージのミスや小さな修正を追加したい場合、`git commit --amend`を使用することができる。ただし、コミットIDが変わるため、すでにpushしている場合は注意する。

直前のコミットメッセージを変更する

```
git commit --amend -m "新しいコミットメッセージ"
```

追加の変更内容を直前のコミットに追加する

```
git add # ファイルをステージングエリアに追加する  
git commit --amend --no-edit
```


コミットの整理

一度コミットした履歴は整理することができる。コミットを整理することで以下の利点がある。

- プロジェクトの変更履歴を理解しやすくする
- コードレビューの効率を向上させる
- バグの原因特定を容易にする
- チーム開発における協力を円滑にする

ただし、すでにリモートリポジトリにプッシュしたコミットは他の人に影響を与える可能性があるため、コミットの整理はチームで相談した上で判断する。

コミット履歴を大幅に変更したい場合(`git reset --soft`)

```
git reset --soft HEAD~N      # 遡りたいコミットまで指定し、コミットを削除
git reset # すれー징エリアを削除
# 以下2つのコマンドを繰り返す
git add -p # 必要な部分のみステージングエリアに追加
git commit -m "新しいコミットメッセージ"
```

既存のコミット履歴を少し変更したい場合(`git rebase -i`)

`git rebase -i HEAD~N` で整理するコミットの範囲を指定する(Nはコミット数)。その次に、エディタが開き、各コミットに対する操作を決定する。上から順に古いコミットが表示される操作内容は以下である(よく使うコマンドを載せている)。

- pick (p): コミットをそのまま適用
- reword (r): コミットメッセージを編集
- edit (e): コミットを編集 (ファイル変更可能)
- squash (s): 前のコミットと統合 (メッセージも統合)
- fixup (f): 前のコミットと統合 (メッセージは破棄)
- drop (d): コミットを削除。そのコミットにおける変更内容も削除される。

コマンド操作例

```
# 直近3つのコミットを整理する
git rebase -i HEAD~3

# 途中でリベースを中止したい場合
git rebase --abort
```

特定にコミットを別ブランチに適用する(`git cherry-pick`)

`git cherry-pick`を行うことで、特定のコミットを別ブランチに適用することができる。例えば、特定のバグ修正をブランチに適用したり、リリースブランチに必要な機能のみを選択的に適用したりする時に使用できる。

```
# 特定のコミットを現在のブランチに適用
git cherry-pick <commit-hash>

# 複数のコミットを適用
git cherry-pick <commit1> <commit2>

# コミット作成せずに変更のみ適用
git cherry-pick --no-commit <commit-hash>
```

行単位の変更履歴の確認(`git blame`)

`git blame` を用いることで、行単位の変更履歴を確認することができる。より詳細に変更履歴を確認したい場合に用いる。

```
# ファイル全体の blame
git blame filename.txt

# 特定の行範囲
git blame -L 10,20 filename.txt

# より詳細な情報
git blame -w -C filename.txt
```